eng-practices

The Standard of Code Review

The primary purpose of code review is to make sure that the overall code health of Google's code base is improving over time. All of the tools and processes of code review are designed to this end.

In order to accomplish this, a series of trade-offs have to be balanced.

First, developers must be able to *make progress* on their tasks. If you never submit an improvement to the codebase, then the codebase never improves. Also, if a reviewer makes it very difficult for *any* change to go in, then developers are disincentivized to make improvements in the future.

On the other hand, it is the duty of the reviewer to make sure that each CL is of such a quality that the overall code health of their codebase is not decreasing as time goes on. This can be tricky, because often, codebases degrade through small decreases in code health over time, especially when a team is under significant time constraints and they feel that they have to take shortcuts in order to accomplish their goals.

Also, a reviewer has ownership and responsibility over the code they are reviewing. They want to ensure that the codebase stays consistent, maintainable, and all of the other things mentioned in "What to look for in a code review."

Thus, we get the following rule as the standard we expect in code reviews:

In general, reviewers should favor approving a CL once it is in a state where it definitely improves the overall code health of the system being worked on, even if the CL isn't perfect.

That is *the* senior principle among all of the code review guidelines.

There are limitations to this, of course. For example, if a CL adds a feature that the reviewer doesn't want in their system, then the reviewer can certainly deny approval even if the code is well-designed.

A key point here is that there is no such thing as "perfect" code—there is only *better* code. Reviewers should not require the author to polish every tiny piece of a CL before granting approval. Rather, the reviewer should balance out the need to make forward progress compared to the importance of the changes they are suggesting. Instead of seeking perfection, what a reviewer should seek is *continuous improvement*. A CL that, as a whole, improves the maintainability, readability, and understandability of the system shouldn't be delayed for days or weeks because it isn't "perfect."

Reviewers should *always* feel free to leave comments expressing that something could be better, but if it's not very important, prefix it with something like "Nit: " to let the author know that it's just a point of polish that they could choose to ignore.

Note: Nothing in this document justifies checking in CLs that definitely *worsen* the overall code health of the system. The only time you would do that would be in an emergency.

Mentoring

Code review can have an important function of teaching developers something new about a language, a framework, or general software design principles. It's always fine to leave comments that help a developer learn something new. Sharing knowledge is part of improving the code health of a system over

time. Just keep in mind that if your comment is purely educational, but not critical to meeting the standards described in this document, prefix it with “Nit: “ or otherwise indicate that it’s not mandatory for the author to resolve it in this CL.

Principles

- Technical facts and data overrule opinions and personal preferences.
- On matters of style, the style guide is the absolute authority. Any purely style point (whitespace, etc.) that is not in the style guide is a matter of personal preference. The style should be consistent with what is there. If there is no previous style, accept the author’s.
- **Aspects of software design are almost never a pure style issue or just a personal preference.** They are based on underlying principles and should be weighed on those principles, not simply by personal opinion. Sometimes there are a few valid options. If the author can demonstrate (either through data or based on solid engineering principles) that several approaches are equally valid, then the reviewer should accept the preference of the author. Otherwise the choice is dictated by standard principles of software design.
- If no other rule applies, then the reviewer may ask the author to be consistent with what is in the current codebase, as long as that doesn’t worsen the overall code health of the system.

Resolving Conflicts

In any conflict on a code review, the first step should always be for the developer and reviewer to try to come to consensus, based on the contents of this document and the other documents in The CL Author’s Guide and this Reviewer Guide.

When coming to consensus becomes especially difficult, it can help to have a face-to-face meeting or a video conference between the reviewer and the author, instead of just trying to resolve the conflict through code review comments. (If you do this, though, make sure to record the results of the discussion as a comment on the CL, for future readers.)

If that doesn’t resolve the situation, the most common way to resolve it would be to escalate. Often the escalation path is to a broader team discussion, having a Technical Lead weigh in, asking for a decision from a maintainer of the code, or asking an Eng Manager to help out. **Don’t let a CL sit around because the author and the reviewer can’t come to an agreement.**

Next: What to look for in a code review

This site is open source. Improve this page.